Chapter 6

Monitoring and reliability (SRE)

6.1 Deployment and capacity (15 points)

Owner: SD. **Status:** deployed and verified November 10, 2026; restart test PENDING.

Baseline and capacity. Before anything was added, the server was read without changes (Listing 41): an EC2 `t3.micro` with 913 MiB of RAM, of which only 200 MiB was available and 425 MiB was already in swap, while Gitea, Jenkins, Caddy and the Actions runner were running. Four more containers would not have fit. BD decided to resize the same instance to `t3.small` (same disk and Elastic IP); afterwards the host had 1,909 MiB with 1,064 MiB available and no swap in use (Listing **??**). The root disk had 8.4 GB free, enough for the images (about 1.4 GB) and Prometheus' capped storage.

What runs. Four separate containers, all from images published by the projects themselves; nothing is built locally and no Dockerfile is written. Tags are pinned to exact releases (never `latest`) and the pulled image digests are recorded (Listing 42 for the choice, Listing 43 for the digests on the server).

Table 6.1: Monitoring containers, versions, and limits

Container	Image	Role, limit, storage
`prometheus`	`prom/prometheus:v3.16.0`	scrapes every 30 s and stores metrics; 256 MiB; volume `monitoring_prometheus_data`, kept 7 days or 1 GB, whichever comes first
`grafana`	`grafana/grafana:13.3.1`	dashboard and alerts; 384 MiB; volume `monitoring_grafana_data` (users, data source, dashboard, alert rules)
`node-exporter`	`quay.io/prometheus/node-exporter:v1.12.1`	whole-host CPU, memory, root filesystem; 64 MiB; host `/` mounted read-only

Container	Image	Role, limit, storage
`blackbox-exporter`	`quay.io/prometheus/blackbox-exporter:v0.29.0`	HTTPS probes of the Gitea and Jenkins login pages; 64 MiB; no storage

The memory limits make sure that, if memory runs out, a monitoring container is stopped rather than Gitea or Jenkins. Grafana's limit started at 256 MiB, but Grafana used 216 MiB while idle, so it was raised to 384 MiB on the first day (recorded in Listing 43). Retention of 7 days follows the course's lab suggestion; the 1 GB cap protects the root disk that Jenkins builds also use.

Configuration in Gitea. The non-secret files are in the `infra` repository, folder `monitoring`, added through issue #1 and pull request #2, reviewed and merged by BD **[TODO: merge commit link and date after BD merges PR #2]**. The same files run in `~/monitoring` on the server. No Compose service publishes a host port, none is privileged, and none mounts the Docker socket.

```yaml
# Monitoring for the Group 1 startup server (course manual Chapter 5). Non-secret; start from ~/monitoring:
#   docker compose config --quiet && docker compose pull && docker compose up -d
# Four separate containers from project-published images. Design for a ~1 GB host shared with Gitea and
↪  Jenkins:
# - NO published host ports: Prometheus 9090, Node Exporter 9100 and Blackbox 9115 live only on the private
#   `monitoring` network; Grafana 3000 is reached by Caddy by service name on the existing `gitea_web` network.
# - No container gets the Docker socket or privileged mode.
# - Memory caps so that a monitoring container, not Gitea or Jenkins, is killed if memory runs out.
# - Named volumes keep Prometheus metrics history and Grafana state across container replacement.
#   Use `docker compose stop/start`; never `docker compose down -v` (deletes the volumes).
services:
  prometheus:
    image: prom/prometheus:${PROMETHEUS_TAG:?choose a release}
    container_name: prometheus
    restart: unless-stopped
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=7d        # course's suggested lab retention
      - --storage.tsdb.retention.size=1GB       # hard cap so the root disk keeps room for builds
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    mem_limit: 256m
    networks: [monitoring]

  grafana:
    image: grafana/grafana:${GRAFANA_TAG:?choose a release}
    container_name: grafana
    restart: unless-stopped
    environment:
      GF_SERVER_ROOT_URL: "https://metrics.ssw590-g1.tech/"
      GF_AUTH_ANONYMOUS_ENABLED: "false"
      GF_USERS_ALLOW_SIGN_UP: "false"
      GF_ANALYTICS_REPORTING_ENABLED: "false"
      GF_ANALYTICS_CHECK_FOR_UPDATES: "false"
      # No admin/admin window on the public login page: the first admin password is a random value generated
      # on the server (secrets/grafana_admin_password in a mode-700 folder, never printed, never in Git). The
      ↪  owner then
      # sets the real one with `docker exec -it grafana grafana cli admin reset-admin-password`.
      GF_SECURITY_ADMIN_PASSWORD__FILE: /run/secrets/grafana_admin_password
    secrets: [grafana_admin_password]
```

```
    volumes:
      - grafana_data:/var/lib/grafana
    mem_limit: 384m                          # 256m left too little room: Grafana 13.3.1 used ~216 MiB idle
    networks: [monitoring, web]

  node-exporter:
    image: quay.io/prometheus/node-exporter:${NODE_EXPORTER_TAG:?choose a release}
    container_name: node-exporter
    restart: unless-stopped
    pid: host                                # host process namespace, for host-wide CPU/memory figures
    command:
      - --path.rootfs=/host
      - --path.procfs=/host/proc
      - --path.sysfs=/host/sys
    volumes:
      - /:/host:ro,rslave                    # read-only host root, for root-filesystem size/free space
    mem_limit: 64m
    networks: [monitoring]

  blackbox-exporter:
    image: quay.io/prometheus/blackbox-exporter:${BLACKBOX_TAG:?choose a release}
    container_name: blackbox-exporter
    restart: unless-stopped
    command: ["--config.file=/config/blackbox.yml"]
    volumes:
      - ./blackbox/blackbox.yml:/config/blackbox.yml:ro
    mem_limit: 64m
    networks: [monitoring]                   # normal bridge egress: probes go out to the public HTTPS
    ↪ hostnames

networks:
  monitoring: {}                             # private: Prometheus <-> exporters <-> Grafana
  web:
    external: true
    name: gitea_web                          # existing network shared by Caddy, Gitea and Jenkins (reused,
    ↪ not recreated)

secrets:
  grafana_admin_password:
    file: ./secrets/grafana_admin_password       # created on the server only; gitignored

volumes:
  prometheus_data:
  grafana_data:
```

Listing 40: `compose.yaml` of the monitoring stack (`infra/monitoring`): four containers, no published ports, memory limits, named volumes, Grafana also on the existing `gitea_web` network.

Start, update, recover. From `~/monitoring` on the server:

```
cp .env.example .env                         # image tags only
docker compose config --quiet && docker compose pull && docker compose up -d
docker compose ps                            # four containers Up
```

To update, change a tag in `.env`, run `docker compose pull && docker compose up -d`, and record the new digests. To stop and start, use `docker compose stop` and `docker compose start`. **Never** `docker compose down -v`: it deletes both volumes, that is, the

metric history and all of Grafana's state. The first Grafana administrator password was a random value generated on the server into `secrets/grafana_admin_password` (folder mode 700, never printed, not in Git), so the default `admin`/`admin` was never valid on the public login page; BD then set her own password with `grafana cli admin reset-admin-password` (Section 6.3).

Deployment diagram. Figure 6.1 adapts the course's Figure 5.1 to this server.

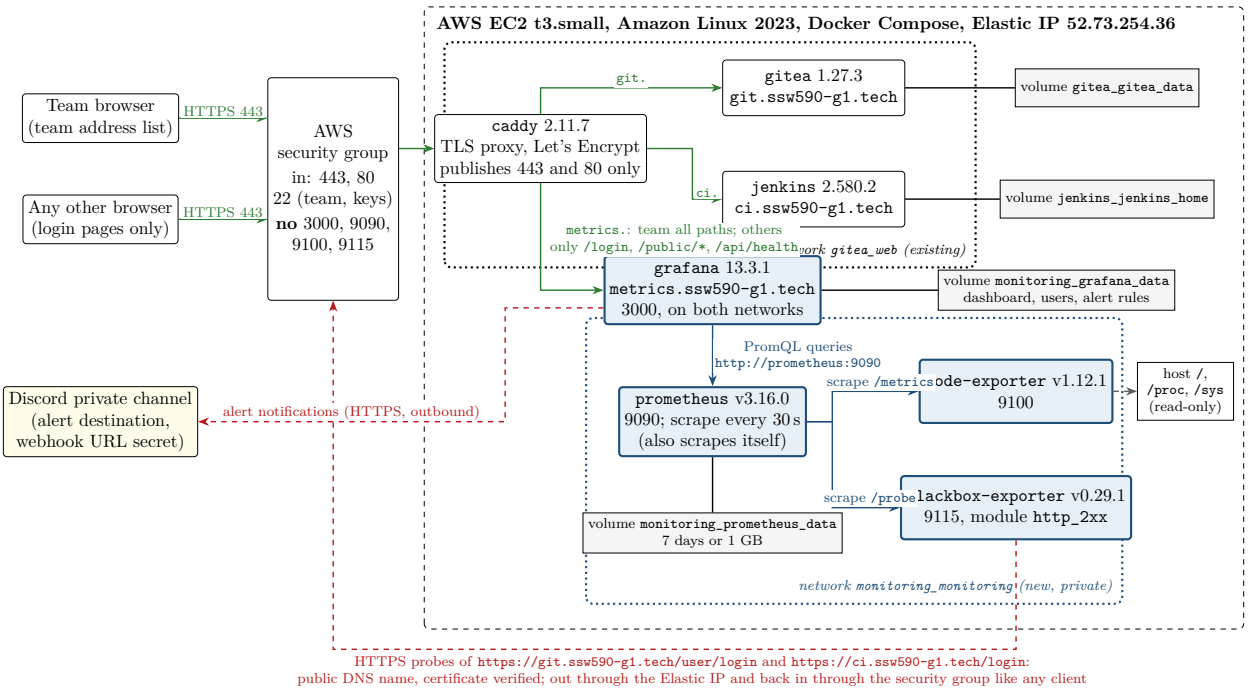


Figure 6.1: Deployment of the Group 1 server with monitoring, adapted from the course manual's Figure 5.1. Blue: the four new monitoring containers. Arrows start at the side that opens the connection. Gitea and Jenkins are probed only through their public HTTPS names; Prometheus, the exporters, and Grafana's port 3000 are reachable only on private Docker networks. Not drawn: the Gitea Actions runner and the Gitea–Jenkins webhook (Chapter 5).

Restart persistence. **[TODO: After BD's controlled outage: restart `prometheus`, then `grafana` (`docker restart`); show the earlier time series, the dashboard, the data source and the alert rules still there, and name the collection gap. Recorded once, cited here and in Section 6.7.]** This shows that the volumes persist; it is not a backup and restore.

```
=== Server baseline before monitoring (read-only), 2026-11-10 23:26:08 UTC ===
instance type: t3.micro  zone: us-east-1c
--- OS / kernel / arch
PRETTY_NAME="Amazon Linux 2023.12.20261014"
6.18.54-121.166.amzn2023.x86_64
```

```
x86_64
--- CPU
2
model name        : Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
--- memory (MiB)
               total        used        free      shared  buff/cache   available
Mem:             913         575          60           0         277         200
Swap:           2047         425        1622
--- root filesystem
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1   20G   12G  8.4G  58% /
/dev/nvme0n1p1 21395124224 12391948288 9003175936  58% /
--- load
 23:26:08 up 26 days,  6:09,  0 users,  load average: 0.03, 0.01, 0.00
--- docker
Docker Engine 25.0.16
Docker Compose version v5.6.0
--- containers
gitea        gitea/gitea:1.27.3        Up 8 days (healthy)        22/tcp, 3000/tcp
caddy        caddy:2.11.7        Up 8 days        0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp,
↪  :::443->443/tcp, 443/udp, 2019/tcp
jenkins        group1/jenkins-python:2.580.2        Up 7 days        8080/tcp, 50000/tcp
gitea-runner        gitea/act_runner:0.6.1        Up 3 weeks
--- container memory (one sample)
gitea        1.57%        110.3MiB / 913.1MiB
caddy        0.00%        20.78MiB / 913.1MiB
jenkins        0.14%        271MiB / 640MiB
gitea-runner        0.11%        14.14MiB / 913.1MiB
--- Docker disk usage
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          5         4         6.669GB   5.818GB (87%)
Containers      4         4         217.8MB   0B (0%)
Local Volumes   6         5         275.3MB   0B (0%)
Build Cache     30        0         1.163kB   1.163kB
--- listening TCP sockets on the host (addresses only)
0.0.0.0:22
0.0.0.0:443
0.0.0.0:80
127.0.0.1:37001
[::]:22
[::]:443
[::]:80
--- service versions
gitea version 1.27.3 built with go1.26.7-X:jsonv2 : bindata, timetzdata,
Jenkins login HTTP 200
x-jenkins: 2.580.2
Gitea login HTTP 200
```

Listing 41: Baseline before monitoring: instance type, memory and swap, disk, containers, listening ports, November 10, 2026 (real output).

```
Image tags and digests chosen for the monitoring stack, read from the registries 2026-11-10 23:27:22 UTC by
↪  Claude Code for SD.
Server architecture: x86_64 (linux/amd64). Index digest = multi-arch list; amd64 digest = what the server will
↪  run.
Also: local validation of the drafted configuration (docker compose config, promtool, blackbox
↪  --config.check).

prom/prometheus:v3.16.0
    index sha256:e04987641b57704f408e31f6373ba4138ecf27cb4a165d24aabb6c38fddd0f06
    amd64 sha256:89c1a644817e9ad70fa2af76c84898e327c1de7063735f962e8313b1e28672c3
grafana/grafana:13.3.1
    index sha256:0615ec4fbd4072004b7d86b46ded63013a74b7d6eb1119273dd89e8fa25c13cf
    amd64 sha256:1c88285f1dd391a62232c010e6c224846e6a73bdc583b3365be2eba614b7919e
```

```
quay.io/prometheus/node-exporter:v1.12.1
    index sha256:1b4e4438faca4dd7e001dd445d161a4a2091b0fededa84093b3a8dfeae1f1be0
    amd64 sha256:da83fae85603c4e47e6c68369a7d746e2dda683dc35ea2e234b4f171e0d92798
quay.io/prometheus/blackbox-exporter:v0.29.1
    index sha256:ea1c7aa3ef3f307cd03e85f34abb604478ccb90b567fc690da0cd9776da76b13
    amd64 sha256:66eae352a79e6f677aeddd8c25a9bdb017586dc5fbd230f22329213d891f94bd

$ docker compose config --quiet
compose config OK
$ promtool check config prometheus.yml
Checking /etc/prometheus/prometheus.yml
 SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax
$ blackbox_exporter --config.check
time=2026-11-10T23:27:14.900Z level=INFO source=main.go:100 msg="(go=go1.27.3, platform=linux/arm64,
↪  user=root@ab03ed60a291, date=20261021-18:03:52, tags=unknown)"
time=2026-11-10T23:27:14.901Z level=INFO source=config.go:144 msg="Configuration file change detected,
↪  reloading the configuration."
time=2026-11-10T23:27:14.901Z level=INFO source=main.go:108 msg="Config file is ok exiting..."
```

Listing 42: Chosen tags with index and linux/amd64 digests, and local validation of the configuration, November 10, 2026 (real output).

```
Real output, Nov 10, 2026 evening US Eastern (server clock UTC, already Nov 11). Assignment 7, rows 1 and 2.
↪  Run by Sakshi (Claude Code over SSH as ec2-user).
Server: t3.small after BD's resize (evidence 140). Files copied from assignment-7/monitoring/ to ~/monitoring;
↪  .env copied from .env.example (tags only).

=== Change made before the first start (security) ===
Brahmi's Caddy block keeps /login public, and the Caddy log already showed scanners on the new name (evidence
↪  141).
So Grafana never started with admin/admin: compose passes GF_SECURITY_ADMIN_PASSWORD__FILE from a Compose
↪  secret,
~/monitoring/secrets/grafana_admin_password = `openssl rand -base64 33` (44 bytes), created on the server,
↪  never printed,
folder mode 700, gitignored. Brahmi replaces it with `docker exec -it grafana grafana cli admin
↪  reset-admin-password`.

=== 2026-11-11 03:14-03:15 UTC: docker compose config --quiet && docker compose pull && docker compose up -d
↪  ===
config OK
 Image prom/prometheus:v3.16.0 Pulled
 Image grafana/grafana:13.3.1 Pulled            (node-exporter and blackbox-exporter pulled likewise)
 Volume monitoring_prometheus_data Created
 Volume monitoring_grafana_data Created
 Network monitoring_monitoring Created
 Container node-exporter Started / prometheus Started / grafana Started / blackbox-exporter Started

$ docker compose ps
NAME               IMAGE                                          STATUS          PORTS
blackbox-exporter  quay.io/prometheus/blackbox-exporter:v0.29.1   Up 20 seconds   9115/tcp
grafana            grafana/grafana:13.3.1                         Up 20 seconds   3000/tcp
node-exporter      quay.io/prometheus/node-exporter:v1.12.1       Up 21 seconds   9100/tcp
prometheus         prom/prometheus:v3.16.0                        Up 20 seconds   9090/tcp
(PORTS shows container ports only; no host bindings.)

=== docker inspect (03:16:20 UTC) ===
/prometheus        restart=unless-stopped mem=268435456 privileged=false ports={} mounts=prometheus.yml(ro),
↪  monitoring_prometheus_data:/prometheus
/grafana           restart=unless-stopped mem=268435456 privileged=false ports={}
↪  mounts=monitoring_grafana_data:/var/lib/grafana, secret file
/node-exporter     restart=unless-stopped mem=67108864  privileged=false ports={} mounts=/:/host (ro,rslave)
/blackbox-exporter restart=unless-stopped mem=67108864  privileged=false ports={} mounts=blackbox.yml(ro)
No container mounts /var/run/docker.sock.
```

```
=== RepoDigests of the pulled images (match the index digests in evidence 111) ===
prom/prometheus:v3.16.0
↪  prom/prometheus@sha256:e04987641b57704f408e31f6373ba4138ecf27cb4a165d24aabb6c38fddd0f06 amd64
grafana/grafana:13.3.1
↪  grafana/grafana@sha256:0615ec4fbd4072004b7d86b46ded63013a74b7d6eb1119273dd89e8fa25c13cf amd64
quay.io/prometheus/node-exporter:v1.12.1
↪  quay.io/prometheus/node-exporter@sha256:1b4e4438faca4dd7e001dd445d161a4a2091b0fededa84093b3a8dfeae1f1be0
↪  amd64
quay.io/prometheus/blackbox-exporter:v0.29.1  quay.io/prometheus/blackbox-exporter@sha256:ea1c7aa3ef3f307cd03
↪  e85f34abb604478ccb90b567fc690da0cd9776da76b13 amd64
`docker buildx imagetools inspect` (SD's Mac) confirms each index's linux/amd64 entry is the amd64 digest in
↪  evidence 111.

=== Prometheus /api/v1/targets (job instance health lastScrape lastError) ===
blackbox   blackbox-exporter:9115                  up 2026-11-11T03:16:08 ''
https      https://git.ssw590-g1.tech/user/login   up 2026-11-11T03:16:16 ''
https      https://ci.ssw590-g1.tech/login         up 2026-11-11T03:16:19 ''
node       node-exporter:9100                      up 2026-11-11T03:16:16 ''
prometheus prometheus:9090                         up 2026-11-11T03:16:12 ''

=== Instant queries (03:16 UTC) ===
probe_success{instance="https://git.ssw590-g1.tech/user/login"}            1
probe_success{instance="https://ci.ssw590-g1.tech/login"}                  1
probe_http_status_code  git 200   ci 200
probe_duration_seconds  git 0.014027061   ci 0.025555427
probe_ssl_earliest_cert_expiry  git 1799858155   ci 1801368269
node_memory_MemTotal_bytes      2002350080
node_memory_MemAvailable_bytes  823209984
node_filesystem_size_bytes{device="/dev/nvme0n1p1",fstype="xfs",mountpoint="/"}   21395124224
node_filesystem_avail_bytes{device="/dev/nvme0n1p1",fstype="xfs",mountpoint="/"}  6994649088
count(node_cpu_seconds_total{mode="idle"})  2

=== Same moment on the host ===
$ free -b
               total        used        free      shared  buff/cache   available
Mem:      2002350080  1031618560    74612736     1048576   896118784   804671488
Swap:     2147479552   170827776  1976651776
$ df -B1 /
Filesystem       1B-blocks        Used  Available Use% Mounted on
/dev/nvme0n1p1 21395124224 14400479232 6994644992  68% /
Comparison: MemTotal equal to free's total; filesystem size equal to df 1B-blocks; avail differs by 4096 bytes
↪  (one block,
taken a few seconds apart); MemAvailable 823 MB vs free 805 MB, sampled ~10 s apart. Root fs is 32.7% free
↪  (68% used,
up from 58% because of the ~1.4 GB of new images).

=== Host listening sockets (sudo ss -ltnp) - unchanged: only 22, 80, 443 public ===
0.0.0.0:80 docker-proxy    0.0.0.0:443 docker-proxy    0.0.0.0:22 sshd    127.0.0.1:41143 containerd   (+ IPv6
↪  [::]:22/80/443)

=== Memory per container (docker stats --no-stream) ===
grafana 210.1MiB / 256MiB   prometheus 45.57MiB / 256MiB   node-exporter 11.23MiB / 64MiB   blackbox-exporter
↪  14.51MiB / 64MiB
gitea 102MiB   caddy 20.49MiB   jenkins 330.7MiB / 640MiB   gitea-runner 10.91MiB

=== Adjustment at 03:18 UTC: Grafana memory cap 256m -> 384m ===
Reason: 216 MiB used at idle, too close to the cap; an OOM kill would also stop alerting. Host available ~780
↪  MiB.
$ docker compose up -d   ->  Container grafana Recreate / Recreated / Started   (volume
↪  monitoring_grafana_data kept)
After restart: grafana 316.8MiB / 384MiB, OOMKilled=false, restarts=0; host available 747 MiB, swap used 165
↪  MiB of 2047.

=== Grafana from inside the network (from the caddy container) ===
GET http://grafana:3000/api/health ->
↪  {"database":"ok","version":"13.3.1","commit":"da3aa3ff574e7dbbf808c66b6daa744defed0441"}
POST http://grafana:3000/login {"user":"admin","password":"admin"} -> HTTP/1.1 401 Unauthorized
```

```
grafana log: msg="Failed to authenticate request" ... error="[password-auth.invalid] invalid password"
```

Listing 43: Deployment: pull and start, container settings, pulled digests, Prometheus targets, first probe results, host values, Grafana limit change, November 10, 2026 (real output).

6.2 Metrics and HTTPS probes (15 points)

Owner: SD. **Status:** all targets up and host values verified November 10, 2026 **PASS**.

What is scraped. Prometheus scrapes four jobs every 30 seconds, all by private service name on the `monitoring` network (Listing 44): itself (`prometheus:9090`), the Node Exporter (`node-exporter:9100`), the Blackbox Exporter's own metrics (`blackbox-exporter:9115`), and the `https` probe job. Grafana reads only from Prometheus; it does not contact Gitea, Jenkins, or the exporters. Right after the start, all five targets were `up` with no scrape error (Listing 43).

```yaml
# Scrape configuration (course manual Chapter 5, step 2.3), adapted to Group 1's real login URLs.
global:
  scrape_interval: 30s
  evaluation_interval: 30s

scrape_configs:
  - job_name: prometheus                          # Prometheus' own metrics
    static_configs:
      - targets: ["prometheus:9090"]

  - job_name: node                                # whole-host CPU, memory, root filesystem
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: blackbox                            # the Blackbox Exporter's own health metrics
    static_configs:
      - targets: ["blackbox-exporter:9115"]

  - job_name: https                               # HTTPS availability probes (multi-target pattern)
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - https://git.ssw590-g1.tech/user/login
          - https://ci.ssw590-g1.tech/login
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115
```

Listing 44: `prometheus.yml`: four jobs, 30 s interval, and the multi-target relabelling that sends each URL to the Blackbox Exporter.

HTTPS probes and what "success" means. The `https` job asks the Blackbox Exporter to fetch `https://git.ssw590-g1.tech/user/login` and `https://ci.ssw590-g1.tech/login`. The relabelling turns each URL into the `target` parameter and the `instance` label, and sends the request to `blackbox-exporter:9115`. A probe counts as successful (`probe_success` = 1) only if the public DNS name resolves over IPv4, the TLS certificate is valid for that name (verification is on), the answer is HTTPS, and the final status after redirects is 2xx, all within 10 seconds (Listing 45). The probe uses the same public names as a user: it leaves through the Elastic IP and comes back in through the security group and Caddy (Figure 6.1). The first probes returned success 1, HTTP 200, in 0.014 s (Gitea) and 0.026 s (Jenkins) (Listing 43).

```yaml
# Probe module (course manual Chapter 5, step 2.3). Success = DNS resolves, TLS certificate valid for the
# hostname (verification ON), at most the default redirects, and a final 2xx response over IPv4.
modules:
  http_2xx:
    prober: http
    timeout: 10s
    http:
      method: GET
      preferred_ip_protocol: ip4
      ip_protocol_fallback: false
      valid_status_codes: []                    # empty = any 2xx
      fail_if_ssl: false
      fail_if_not_ssl: true                     # an http:// answer would count as failure
      tls_config:
        insecure_skip_verify: false
```

Listing 45: `blackbox.yml`: module `http_2xx` with TLS verification on and plain HTTP counted as failure.

Metric inventory.

Table 6.2: Metrics used in this chapter

Metric	Type	Meaning and limitation
`node_cpu_seconds_total`	counter	CPU seconds per core and mode since boot; only meaningful through `rate()`; whole host, not per container
`node_memory_MemAvailable_bytes`	gauge	the kernel's estimate of memory that programs can still get (not "free" memory)
`node_filesystem_avail_bytes`, `node_filesystem_size_bytes`	gauge	bytes usable by normal users, and size, per filesystem; the root filesystem must be selected by its labels
`up`	gauge	1 if Prometheus' last scrape of a target worked; says nothing about the probed service
`probe_success`	gauge	1 if the whole HTTPS check passed; a login page only, not a Git push or a build

Metric	Type	Meaning and limitation
`probe_duration_seconds`	gauge	time of the whole probe, including DNS, TLS, and redirects

Host values checked against the operating system. At 03:16 UTC on November 11 (evening of November 10, Eastern time), the Prometheus values were compared with `free -b` and `df -B1 /` run on the host (Listing 43). The root filesystem series has `mountpoint="/"`, `device="/dev/nvme0n1p1"`, `fstype="xfs"`; overlay and tmpfs mounts are excluded, never averaged in.

Table 6.3: Prometheus compared with the host

Value	Prometheus	Host command	Result
Total memory	2,002,350,080	`free -b`: 2,002,350,080	equal
Available memory	823,209,984	`free -b`: 804,671,488	2 % apart; sampled about 10 s apart
Root size	21,395,124,224	`df -B1`: 21,395,124,224	equal
Root available	6,994,649,088	`df -B1`: 6,994,644,992	one 4 KiB block apart
CPUs	2 idle series	2 vCPUs (`t3.small`)	equal

Root free space was therefore 32.7 % (68 % used, up from 58 % because of the new images), well above the 15 % alert threshold of Section 6.6.

Limitations. A 30-second scrape cannot see events shorter than about 30 seconds. The probes run on the same server they check, so a total server outage stops the probes and the alerts too; an independent outside check is proposed in Section 6.7. A probe from AWS does not prove that every user network can reach the services. Node Exporter reports the whole host; per-container usage (cAdvisor) is outside the required scope.

6.3 Access control and exposure (10 points)

Owner: SD (Caddy block BD, accounts BD). **Status:** tested from a team and a non-team network **PASS**.

Login URLs. All three services use HTTPS with Let's Encrypt certificates; no password is given here (see the Hosts and Passwords chapter for where each is kept):

- Gitea: `https://git.ssw590-g1.tech/user/login`

- Jenkins: https://ci.ssw590-g1.tech/login
- Grafana: https://metrics.ssw590-g1.tech/login (certificate issued November 11, 2026, Listing 47)

How Grafana is reached. The DNS name `metrics.ssw590-g1.tech` points to the Elastic IP (Listing **??**). Caddy, the existing TLS proxy, is the only container with public ports (443 and 80). It forwards to `grafana:3000` on the private `gitea_web` network. The block follows the Jenkins rule: addresses on the team list (`TEAM_IPS`, kept only in the server's `.env`) reach all of Grafana; everyone else reaches only the login page, its static files, and the health check; everything else gets `403 Forbidden`.

```
# The metrics.ssw590-g1.tech site block, now part of the main Caddyfile (applied by BD on Nov 10, 2026; IT
↪  manual evidence 141).
# Same rule as the ci. block: everyone may open ONLY Grafana's login page and the static files it needs;
↪  everything
# else is limited to TEAM_IPS (server .env only, never in the repo). Grafana is reached by service name on
↪  gitea_web.
# Verify with `caddy validate` before reloading; tested from a non-team network (evidence 114): /login 200, /
↪  403.
metrics.ssw590-g1.tech {
        @team remote_ip {$TEAM_IPS}
        @loginpage path /login /public/* /api/health /favicon.ico

        handle @team {
                reverse_proxy grafana:3000
        }
        handle @loginpage {
                reverse_proxy grafana:3000
        }
        handle {
                respond "Forbidden" 403
        }
}
```

Listing 46: Caddy site block for Grafana, as applied on the server (`infra/monitoring/Caddyfile.metrics-block.txt`).

Authentication. Grafana requires a sign-in: anonymous access and self sign-up are off (Listing 40). The administrator account belongs to BD; its first password was random and server-only, and BD replaced it (Figure **??**). SD has her own named account with the Editor role, not an administrator (Listing **??**); her temporary password was handed over privately and changed at first sign-in **[TODO: date SD changed it]**. A grader account is not needed: the dashboard and alert evidence is shown in Sections 6.4 and 6.6.

Exposure tests. The security group is unchanged for monitoring: only 443, 80, and 22 (team addresses, key login only) are open. Ports 3000, 9090, 9100 and 9115 were closed from the internet (Listing 47, Listing 48). The decisive test was from a network that is on no team list, a T-Mobile mobile connection (Listing 48):

Table 6.4: Grafana from a non-team network, November 11, 2026

Request	Result	Meaning
`/login` and its 9 static files	200	the login page works for anyone, certificate verified
`/api/health`	200	health check only
`/`, `/explore`, `/d/...`	403	no dashboards or queries
`/api/datasources`, `/api/user`, `/api/search`, `/api/admin/users`	403	no API
`/metrics`, alert API	403	Grafana's own metrics and alerts hidden
`POST /login` as `admin`/`admin`	401	the default password never worked
TCP 22, 3000, 9090, 9100, 9115	closed	SSH limited to team addresses; monitoring ports private

Limitations. The login form is public, so anyone can try passwords; Grafana's built-in protection against repeated failed logins applies. The campus network's address is on the SSH and Caddy team lists from November 11, 2026 until teardown; it is shared by everyone on campus, so it narrows access less than a personal address. SSH still needs a key, and Grafana still needs a password.

```
Real output, Nov 10, 2026 evening US Eastern (03:16-03:20 UTC Nov 11). Assignment 7, row 3 (access control).
↪  Run by Sakshi.
Caddy block as applied by Brahmi (evidence 141): team addresses get all of Grafana; everyone else only
/login, /public/*, /api/health, /favicon.ico; everything else answers 403.

=== Certificate (openssl s_client from SD's Mac) ===
subject=CN=metrics.ssw590-g1.tech
issuer=C=US, O=Let's Encrypt, CN=YE1
notBefore=Nov 11 01:52:00 2026 GMT   notAfter=Feb  9 01:51:59 2027 GMT
curl ssl_verify_result = 0 on every request below.

=== Team view: SD's Mac (public IP is on the TEAM_IPS list) ===
/login              200  (<title>Grafana</title>)
/                   302 -> /login           (not signed in: login is required)
/public/build/      302 -> /login?redirectTo=...
/api/health         200
/api/datasources    401   (needs sign-in)
/api/admin/users    401
/metrics            200   (Grafana's own metrics; team only, see below)

=== Non-team view: request from the server itself, source = the Elastic IP, which is not on TEAM_IPS ===
/login                                  200
the 9 /public/build/*.js|css files the login page references   200 (all)
/public/img/grafana_icon.svg            200
/favicon.ico                            302
/api/health                             200
/                                       403
/api/datasources                        403
/metrics                                403
/api/admin/users                        403
/api/user                               403
/d/x                                    403
```

```
/explore                                403
/api/login/ping                         403
POST /login {"user":"admin","password":"admin"}   401   (default password never active, evidence 112)

=== Ports scanned from SD's Mac to 52.73.254.36 (nc -z, 4 s timeout) ===
3000 closed/filtered   9090 closed/filtered   9100 closed/filtered   9115 closed/filtered
(security group unchanged; containers publish no host ports, evidence 112)

=== Still to do ===
Phone on mobile data (not the team network): /login loads with a valid certificate; / and /explore show
↪  Forbidden. Screenshot -> next evidence number.
After Brahmi sets the admin password: sign in over HTTPS with SD's own (non-admin) account; record roles.
```

Listing 47: Grafana HTTPS: certificate, team view, simulated non-team view, closed ports, November 10, 2026 (real output).

```
Real output, Nov 11, 2026 19:20 EST (2026-11-12 00:20:54 UTC). Assignment 7, row 3 (access control). Run by
↪  Sakshi from her Mac
on a mobile network (T-Mobile USA), which is NOT on the Caddy team list or the security group's SSH list.
↪  Replaces the planned phone test:
same condition (outside the team network), but with exact status codes instead of screenshots.

client IP: 172.59.210.63  (whois org: T-Mobile USA, Inc.)
login title: <title>Grafana</title>
login page assets: 9/9 returned 200
/login                                        200 verify=0
/api/health                                   200 verify=0
/                                             403 verify=0
/explore                                      403 verify=0
/d/x                                          403 verify=0
/api/datasources                              403 verify=0
/api/user                                     403 verify=0
/api/search                                   403 verify=0
/metrics                                      403 verify=0
/api/admin/users                              403 verify=0
/api/alertmanager/grafana/api/v2/alerts       403 verify=0
POST /login admin/admin                       401
body of /explore: Forbidden
tcp 22: closed/filtered      (SSH also refused from this address: "ssh: connect to host git.ssw590-g1.tech
↪  port 22: Operation timed out")
tcp 3000: closed/filtered
tcp 9090: closed/filtered
tcp 9100: closed/filtered
tcp 9115: closed/filtered

Reading: from outside the team, the only Grafana surface is the login page (with its static files) and the
↪  health check;
dashboards, Explore, the API, Grafana's own /metrics and the alert API are refused by Caddy before reaching
↪  Grafana.
verify=0 means curl verified the Let's Encrypt certificate chain and hostname.
Limitation: the login form itself is public, so a non-team user can attempt passwords; Grafana's default
↪  brute-force
login protection applies, and admin/admin was never valid (evidence 112).
```

Listing 48: Grafana from a non-team mobile network (T-Mobile): only the login page is reachable, November 11, 2026 (real output).

6.4 Dashboard (15 points)

6.5 SLI, SLO, and error budget (10 points)

Owner: SD. **Status:** definitions and worked example written; real short window **PENDING**.

Definitions.

Term	Meaning here
SLI	*Service-level indicator*: a measured fraction. Ours is the share of scheduled HTTPS probes that succeed for one service over a stated window: successful `probe_success` samples divided by all samples received.
SLO	*Service-level objective*: our internal target for that SLI, at least 99 % probe success over a 24-hour window, evaluated separately for Gitea and Jenkins. It is a lab target chosen by the course, not a promise.
SLA	*Service-level agreement*: a commitment to a customer, usually with consequences if missed. This startup has no customer SLA; nothing here is one.

How the SLI is computed. For one service and a window `W`, with one sample every 30 seconds:

```
100 * avg_over_time(probe_success{job="https",
↪  instance="https://ci.ssw590-g1.tech/login"}[W])
count_over_time(probe_success{job="https",
↪  instance="https://ci.ssw590-g1.tech/login"}[W])   # samples received
```

`probe_success` is 1 or 0, so its average over the window is the success fraction. The count is compared with the expected number of samples (`W` / 30 s); a missing sample is unknown, not a success. Dashboard panels 4 and 5 (Section 6.4) show the same series sample by sample.

What this SLI does not see. It measures availability of the login page, not speed. A probe that takes 4 seconds but returns 200 still counts as a success: the Jenkins probe did exactly that once on Nov 11 near 16:25 EST (dashboard panel 6), and the availability SLI did not change. Catching slowness would need a second, latency SLI, for example the share of probes faster than 1 second. A login page that loads also does not prove that a Git push or a build works.

The difference in one line: the SLI is what we measure, the SLO is the target we hold ourselves to, and an SLA is what we would owe someone else if we missed it [26].

Worked example (hypothetical numbers from the course manual, not our results). A probe every 30 seconds for 24 hours gives $24 \times 60 \times 2 = 2{,}880$ scheduled probes per service.

- **Gitea:** 2,880 successes out of 2,880 probes, so the SLI is $100 \times 2880/2880 = 100\,\%$. It meets the 99 % SLO.
- **Jenkins:** 2,850 successes and 30 failures, so the SLI is $100 \times 2850/2880 \approx 98.96\,\%$. It misses the SLO, by about 0.04 percentage points.
- **Error budget:** a 99 % SLO allows 1 % of probes to fail: $0.01 \times 2880 = 28.8$ failed samples. Because each sample stands for 30 seconds, that is $28.8 \times 0.5 = 14.4$ minutes of failure in the day, if failures are spread evenly across samples.
- **Jenkins over budget:** 30 failures use $30 - 28.8 = 1.2$ samples (about 0.6 minutes) more than the budget. Samples are whole, so at most 28 failed samples ($100 \times 2852/2880 \approx 99.03\,\%$) would still meet the target; 29 would not ($\approx 98.99\,\%$).

The budget is a way to decide, not just a score: while Jenkins has budget left, the team can accept risky changes; once it is spent, reliability work comes first.

Our short window. **[TODO: after BD's controlled Jenkins outage: window start/end and time zone, scrape interval 30 s, query evaluation time, `100 * avg_over_time(probe_success{job="https", instance=...}[..])` and `count_over_time(...)` for both services, received vs. expected samples (about 20 per 10 minutes), missing data treated as unknown (not success), and why a short window cannot establish the 24-hour SLO.]**

6.6 Alerts (15 points)

6.7 Controlled incident and review (10 points)

6.8 Manual changes in this version (10 points)